Available online at www.sciencedirect.com

ScienceDirect

Procedia Computer Science 220 (2023) 275–282

Procedia Computer Science

www.elsevier.com/locate/procedia

The 14th International Conference on Ambient Systems, Networks and Technologies (ANT)
March 15-17, 2023, Leuven, Belgium

Deep Reinforcement Learning Based Traffic Signal Control: A Comparative Analysis

Chunliang Wu[a], Inhi Kim[a,*], Zhenliang Ma[b]

[a]*Cho Chun Shik Graduate School of Mobility, Korea Advanced Institute of Science and Technology, Daejeon, 34051, South Korea*
[b]*Department of Civil and Architectural Engineering, KTH Royal Institute of Technology, Stockholm, 11428, Sweden*

Abstract

More recently, the advancement of deep learning techniques enables reinforcement learning (RL) to handle high-dimensional decision-making problems. It brings increasing attention in transport areas. Some work has applied them to solve the intractable traffic signal control (TSC) problem and achieved promising results. However, very few research comprehensively investigates the impacts of key design elements, including state definitions, reward functions and deep reinforcement learning (DRL) methods, on TSC performance. To fill this research gap, this paper first selects commonly used design elements from existing literature. Then, we compare their learning stability and control performance at an isolated intersection under different scenarios via simulation experiments. The experimental results show that the quantitative state (e.g., the number of vehicles on lanes) and image-like state (e.g., vehicle position and speed) have no significant impact on the performance under different traffic demands. However, the impact of various reward functions on performance is vivid, especially under high traffic demand. Also, high-resolution vehicular network data may not be possible to improve control performance versus ordinary camera data. In addition, the value-based DRL algorithms outperform the policy-based algorithm and traditional TSC control methods. The findings of this research would provide insights and guidance for transport engineers to design an efficient DRL-based TSC system in a real traffic environment.

© 2023 The Authors. Published by Elsevier B.V.
This is an open access article under the CC BY-NC-ND license (https://creativecommons.org/licenses/by-nc-nd/4.0)
Peer-review under responsibility of the scientific committee of the Conference Program Chairs

Keywords: Traffic signal control (TSC), Reinforcement learning (RL), Deep learning, Simulation;

1. Introduction

Traffic congestion is on the rise all around the world given population growth and ongoing urbanization. It has negative impacts on quality of life, urban environment and economy. Considering the high investment, optimizing existing available road infrastructure is more cost-efficient and conductive to improve mobility than building new ones. Traffic lights are an integral part of urban road infrastructure controlling traffic flows. According to a report from National Transportation Operations Coalition, inefficient traffic signal timings contribute an estimated 5% to

* Corresponding author. Tel.: +82-42-350-1760
E-mail address: inhi.kim@kaist.ac.kr

1877-0509 © 2023 The Authors. Published by Elsevier B.V.
This is an open access article under the CC BY-NC-ND license (https://creativecommons.org/licenses/by-nc-nd/4.0)
Peer-review under responsibility of the scientific committee of the Conference Program Chairs
10.1016/j.procs.2023.03.036

10% of all traffic related delay [1]. Therefore, developing smart traffic signal control (TSC) systems is of paramount importance to mitigate traffic congestion and improve travel efficiency.

In the past, the most common way used by traffic engineers to optimize traffic signal timings is that they formulate the TSC problem as an optimization problem with certain traffic dynamics assumptions [2]. Although these assumptions allow optimization problems to be tractable, some of them detach from reality. They did not consider many complex and stochastic scenarios of real world; thus, they are less likely to respond to dynamic traffic demands in a timely manner. Thanks to the rapid development of artificial intelligence, a promising learning-based paradigm, called reinforcement learning (RL), has emerged to complement the limitations of model-based control methods [3]. More recently, the great success of deep learning (DL) makes it possible for RL to address high-dimensional issues. An increasing number of studies leveraged the deep reinforcement learning (DRL) framework to address intractable TSC tasks and found it fits well with dynamic traffic environment. These studies used various design elements, including state definitions, reward functions and DRL algorithms. Nevertheless, very few research comprehensively investigates the impacts of these design elements on TSC performance, which to some extent hinders their application in practice [8]. There are some open questions. For example, what types of real data obtained from road sensors and devices can be defined as the state and reward variables to make TSC have better performance. Also, what DRL algorithm is more stable and performs better for TSC tasks. Thus, this study systematically evaluates the impacts of these key design elements on control performance under different traffic demand via isolated intersection simulation experiments.

The remainder of this paper is organized as follows. Section 2 reviews the related works. Section 3 introduces the method used in this work. Section 4 describes the simulation experiments. Section 5 presents and discusses the experimental results. Finally, Section 6 concludes the main findings of this work.

2. Related works

This section summarizes the state and reward design and RL algorithm selection in the relevant TSC literature.

State is to describe the traffic environment that can be learned by the agent; thus, its design is important for the performance of RL-based TSC. In the early stage, RL can only handle low-dimensional data, which allows researchers to use quantitative indicators to characterize the traffic environment. These indicators include queue length, waiting time, travel delay and arrival rate of vehicle [4, 5, 6]. Among them, queue length and arrival rate of vehicle can be easily obtained by loop detector and low-resolution cameras. However, accurate waiting time and travel delay need to aggregate each vehicle travel information collected from vehicular network. In addition, phase duration and signal phases are often defined as state information [7]. Recently, with the aid of deep learning, RL is capable of learning high-dimensional data. Some studies attempted an image-like state representation method for a more comprehensive description of the traffic environment [8, 11]. These studies normally use image format to store vehicle information, such as position and speed.

Reward represents the learning objective of RL-based traffic signal controllers. In the literature, the average travel delay, average waiting time, and queue length are often considered as reward functions [9]. Also, some studies take maximization of throughput and minimization of stops as reward functions [4]. In terms of RL methods, there are two types of model-free RL algorithms, including value-based and policy-based methods, often used for the design of traffic signal controllers [8]. Two classic value-based methods, namely Q-learning and state-action-reward-state-action (SARSA), were often used in the earlier studies. *EI-Tantawy et al.* [4] comprehensively investigated the impacts of these two algorithms on the performance of TSC. In the recent years, deep learning is endowed with strong predictive ability. Thus, a series of algorithms integrating deep neural networks with valued-based RL are developed. The most popular algorithms used for TSC tasks are deep Q-network (DQN), double DQN and duelling DQN [10, 11]. Compared with value-based methods, earlier policy-based method is unlikely to evaluate control decisions at each time step; thus, a very small number of research used it to solve TSC problems. With the emergence of a new type policy-based method, called Actor-Critic, which combines the advantage of classical policy-based and value-based method, some studies attempted to utilize it to the TSC field [8].

3. Methodology

We use the Markov decision process (MDP) framework to model TSC problems at isolated intersections. The MDP is defined by a tuple $< S, A, P, R, \gamma >$, where S is the state space, A the action space, P the state transition dynamic

model, R the reward function and $\gamma \in [0, 1]$ the discount factor. At a time step $t \in T$, the agent I receives the traffic state information $s_t \in S$ from the environment. Given state s_t, the agent takes an action $a_t \in A$ based on a policy π. The traffic environment transitions from state $s_t \in S$ into a new state $s_{t+1} \in S$ according to the state transition dynamics $P(s_{t+1}|s_t, a_t)$, and the agent receives a reward $r_t \in R$. The goal of the learning-based agent I is to find an optimal policy π^* that maximizes the expected cumulative rewards $E[\sum_t^T \gamma^t R(s_t)]$ from time step t onward. The elements of MDP for the TSC problem, including state space, action space, and reward function, are defined as follows.

3.1. State space

Based on the aforementioned literature, two types of state definitions are considered in this study.

State represent 1 (quantitative): The quantitative state consists of two components. The first component is the current signal timing at time step t, which includes the current signal phase p_t and green phase duration g_t. The second component is the total number of vehicles on each lane that can be collected by cameras. Considering the cameras' view distance, we assume it only identifies vehicles on a section of road, such as vehicles within 150 m of the isolated intersection. We use a vector $V = [vl_t^l]_{l \in L}$ to represent this variable, where vl_t^l is denoted as number of vehicles on the lane $l \in L$ at time step $t \in T$.

State represent 2 (image-like): The incoming links of an intersection are divided into multiple cells with the same lengths. In this study, the length of each cell is set to be 5 m considering each cell can only fill in one vehicle information. To better compare with quantitative state definition, it also focuses on vehicles within 150 m of the intersection. We assume the intersection installs the vehicular network, which could collect vehicle information, including position and speed. These information will be stored in an image-like state space consisting of two $C \times L$ vectors, where C represents the number of cells per lane, and L is the number of lanes.

3.2. Action space

The phase sequence is pre-determined. The agent has two actions to select and its action space is defined as $A = \{0, 1\}$. The agent I keeps the current signal phase if it selects an action $a_t = 0$; otherwise, it changes to the next signal phase. Also, a reasonable interval of each time step $\triangle t$ is considered to protect against the frequent change of signal phases and improve the learning stability.

3.3. Reward functions

The reward function determines the convergence speed of RL algorithms and the control performance. In this study, we consider four reward functions.

Function 1 (queue length): To minimize the total queue length, the immediate reward is defined as follows.

$$r_t = -\sum_{l \in L} q_{t+\triangle t}^l, \tag{1}$$

where $q_{t+\triangle t}^l$ is queue length on the lane $l \in L$ at time $t + \triangle t$.

Function 2 (waiting time): To minimize the waiting time of all approaching vehicles, the reward is defined as the difference between the total vehicle waiting time between two decision points.

$$r_t = -\sum_{l \in L}(\sum_{v \in V_{t+\triangle t}^l} w_{t+\triangle t}^v - \sum_{v \in V_t^l} w_t^v), \tag{2}$$

where $w_{t+\triangle t}^v$ is the waiting time of vehicle $v \in V_{t+\triangle t}^l$ at time $t + \triangle t$, and $V_{t+\triangle t}^l$ represents all vehicles on the lane $l \in L$ at time $t + \triangle t$.

Function 3 (throughput): To maximize the number of vehicles passing through the intersection, the reward is denoted as follows:

$$r_t = \sum_{l \in L} vp_{t+\triangle t}^l, \tag{3}$$

where $vp^l_{t+\triangle t}$ is the number of vehicles on the lane $l \in L$ that cross the intersection at time step $t + \triangle t$.

Function 4 (throughput and approaching vehicles): It is not sufficient to only consider maximizing throughput without considering the current approaching vehicles. Thus, a new reward definition that achieves these two objectives simultaneously as follows.

$$r_t = \sum_{l \in L} (vp^l_{t+\triangle t} - vl^l_{t+\triangle t}), \tag{4}$$

where $vl^1_{t+\triangle t}$ is the number of approaching vehicles on the lane $l \in L$ at time $t + \triangle t$.

3.4. Algorithms

3.4.1. Value-based methods

A model-free RL algorithm, called Q-learning, is often applied to solve the above RL problem. The Q-learning uses a Q-value function $Q(s, a)$ to evaluate how good an action is in a given state. The Q-value function is updated as follows [12]:

$$Q(s, a) \leftarrow Q(s, a) + \alpha(r + \gamma \max_{a'} Q(s', a') - Q(s, a)), \tag{5}$$

where s and s' are the current and next state, a the current action, r an immediate reward, $\max_{a'} Q(s', a)$ the maximum attainable reward for the next state, $\alpha \in [0, 1]$ a learning rate and $\gamma \in [0, 1]$ a discount factor. A higher learning rate α means Q-values are updated quickly, and vice versa. Also, a higher discount factor γ indicates that the agent concerns more about long-term future rewards.

However, for most of control problems, storing the values of all state-action pairs in a table is impractical. Thus, *Minh et al.*[13] proposed to use deep neural network (DNN) as a function approximator to estimate the Q-functions. They called this novel algorithm deep-Q-network (DQN). DQN has two separate networks. One is the main Q network that is to approximate Q-function. Another is the target Q network that has the same structure of the main network and guides its update process. The training process of DQN is to minimize the loss function as follows.

$$L(\theta) = E_{s,a}[(y^{DQN} - Q(s, a; \theta))^2], \tag{6}$$

where $y^{DQN} = r + \gamma \max_{a'} Q(s', a'; \theta^-)$ is the target value, $Q(s, a; \theta)$ the main Q network and $Q(s', a'; \theta^-)$ the target Q network. To stabilize the learning process and reduce the correlation between the target and estimated Q values from the main network, the parameters θ^- of the target Q network are fixed for several steps and then copy the parameters from the main Q network, so then $\theta^- = \theta$. Also, to make the network updates more stable, it introduces an experience replay technique that is to store the previous experience (s, a, r, s') into replay memory and randomly select a mini-batch of experience from this memory to train the DNN. In addition, this research adopts an ϵ-greedy approach that randomly selects an action with a probability ϵ, and selects an action with a probability 1-ϵ that will receive the highest estimated reward.

In the DQN algorithm, the best action is selected based on the target Q network. However, the agent does not have enough information about the best action at the beginning of the training; thus, if taking the maximum Q value from the target Q network as the best action might lead to that the target value $r + \gamma \max_{a'} Q(s', a'; \theta^-)$ is overestimated. *Van et al.*[14] proposed a novel algorithm, called double DQN (DDQN), to solve this problem. The target value is written as follows.

$$y^{DDQN} = r + \gamma Q(s', arg\max_{a'}(Q(s', a'; \theta)), \theta^-). \tag{7}$$

The DQN algorithm uses a single DNN to directly predict the value of all possible actions. Note that the value of an action relies on the value of the state and the advantages of taking that action. *Wang et al.*[15] proposed an algorithm, called Dueling DQN, that combine this idea into a DQN framework. The Q-value is estimated using two separate streams of fully connected layers. One is to estimate the state-value function $V(s; \theta, \beta)$ and another stream $A(s, a; \theta, \alpha)$ to estimate the advantage function for action a. Here, θ denotes the sharing parameters of the early fully connected layer, and α and β are the parameters of the two streams of fully connected layers. The Q-value function

can be written as:

$$Q(s,a;\theta,\alpha,\beta) = V(s;\theta,\beta) + (A(s,a;\theta,\alpha) - \frac{1}{|A|}\sum_{a'} A(s,a';\theta,\alpha)), \tag{8}$$

where $\frac{1}{|A|}\sum_{a'} A(s,a';\theta,\alpha)$ is the average advantage of all actions.

3.4.2. Policy-based methods

In addition to the above three value-based methods that use value function $Q(s,a)$ to estimate the performance of all possible actions, policy-based methods that directly learn the policy π_θ with a parameterized function $\pi_\theta(a|s)$ are another type of RL method that often use to solve continuous control problems. One of classical policy-based RL methods is called REINFORCE. It updates the policy parameter θ using policy gradient algorithms as follows [12].

$$\theta \leftarrow \theta + \alpha\nabla_\theta log\pi_\theta(a_t|s_t)R_t, \tag{9}$$

where α is the learning rate and R_t denotes $\sum_{t=0}^{T}\gamma^t r_{t+1}$. Since the R_t has a very high variance, it results in slow convergence. The advantage actor-critic (A2C) algorithm uses the advantage function A_t that has mentioned in the Dueling DQN algorithm to replace R_t to reduce the variance. In this study, we use temporal difference (TD) error to estimate the advantage function as follows.

$$A_t = r_t + \gamma V_\psi(s_{t+1}) - V_\psi(s_t), \tag{10}$$

where the baseline $V_\psi(s_t)$ is the state-value function at the current state. The A2C algorithm includes two networks: Actor and Critic. The Critic is to estimate the value of state $V_\psi(s)$ and the Actor is to decide the agent's behavior based on the policy ϕ_θ. In recent years, DNN is used to parameterize both the Actor and Critic functions [8].

4. Experiments

4.1. Experiment design

The experiments are conducted in a micro-simulation environment in VISSIM [16]. The VISSIM is an advanced and flexible microscopic traffic simulation software, which provides an additional interface, named COM, through which users are able to remotely control simulation by programming. This study uses a Python script to program DRL. Via the COM, DRL controllers can obtain the real-time traffic state information and send control policies to the traffic simulator to control traffic signals. The DRL-based traffic signal controller is tested on an isolated intersection and the geometry of this intersection is shown in Fig. 1. Each approach has three lanes with one straight lane, one left-turn lane and one lane for straight-going and right-turn. The intersection is controlled by four signal phases and its sequence is predetermined.

The low and high traffic demand scenarios are tested in this study. For the low traffic demand, the input demand of each approach is between 600-900 veh/hr. The input demand for the high traffic demand scenario is between 1200-1500 veh/hr. The left-turning and right-turning vehicles account for 20% and 5%, respectively. The DRL models are trained for multiple episodes and each episode is simulated for 3600s with 5 mins warm-up time. Also, as mentioned in Section 3.2, the signal controller needs to select an action every $\triangle t$ seconds. If $\triangle t$ is set too short, it will consume a large amount of computational resources. On the contrary, too long $\triangle t$ will lead that the controller cannot adjust the strategy in time. Based on existing studies [5], the time interval of each $\triangle t$ is set to be 5s. Two widely used traffic signal control methods, including fixed-time control and actuated control, are compared with DRL-based methods.

4.2. Parameter setting

The three value-based DRL methods mentioned in Section 3.4.1 consist of two fully connected layers. For the quantitative state, it is first input into a fully connected layer with 40 hidden units and then is fed into a layer with 30 hidden units. For the image-like state, it is input into two convolutional layers to process the image-like data before putting them into two fully connected layers with the same hidden units as the quantitative state. The first convolutional

Table 1: Parameters of value-based DRL algorithms.

Parameter	Value
Replay memory capacity Z	10000
Mini-batch size B	32
Learning rate α	0.0001
Discount factor γ	0.85
Initial exploration probability ϵ	0.99
Final exploration probability ϵ	0.01
Decay rate ϵ_d	0.0001

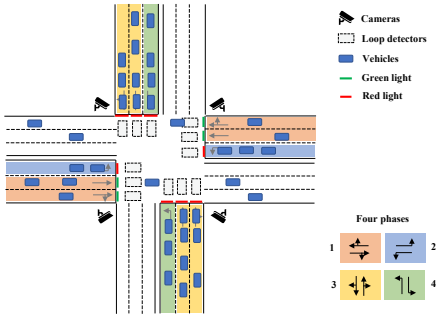


Fig. 1: Illustration of the TSC problem for isolated intersection

layer contains 16 filters. The filter size is 4×4 and it moves 2×2 stride. The second convolutional layer has 32 filters with the size of 2×2 and the stride's size is 1×1. Table 1 summarizes the parameters of value-based DRL algorithms. For the A2C algorithm, the network structure of actor and critic is the same with two fully connected layers. Each layer has 30 hidden units. The learning rate α for actor and critic is set to be 0.0003 and 0.0001, respectively.

5. Simulation results and discussions

5.1. Impacts of state definitions

To analyze the impact of state definitions (Section 3.1) on the performance of TSC, we first apply the DQN algorithm with reward function 1 (queue length) as the base case to train the agent. Table 2 shows the performance of the agent with different state definitions under low and high demand. The results is surprising as there is no significant difference in the control performance by using quantitative and image-like state definitions. This finding is inconsistent with the proposition of DRL-based studies that better performance can be achieved by using more detailed state representation. It implies that in practice, the low-resolution data from camera is enough to guarantee the performance of DRL-based signal controllers rather than installing costly vehicular networks to collect accurate vehicle data.

Table 2: The performance of the agent with different state definitions.

State definitions	Traffic demand	Average delay (sec)	Average speed (km/h)	Total number of stops
1	Low	16.59 ± 0.41	25.60 ± 0.13	1283 ± 23
	High	35.70 ± 0.68	20.57 ± 0.14	3868 ± 63
2	Low	16.60 ± 0.43	25.59 ± 0.14	1284 ± 24
	High	35.82 ± 0.75	20.53 ± 0.16	3866 ± 73

5.2. Impact of reward functions

Since the performance of quantitative and image-like state definitions is similar and the training complexity of image-like state is higher than the quantitative state (mentioned in Section 4.2), we use the quantitative state and DQN algorithm as the baseline to evaluate the impact of different reward functions on control performance. Table 3 shows that for the low traffic demand, four rewards have no significant impact on the performance of the agent in terms of average delay, average speed and total number of stops. However, the performance of the reward function 4 is better than other rewards under high traffic demand. It indicates that only considering queue length or throughput as the reward is not enough for the agent to learn optimal control policy, especially for the congested scenarios. Also, compared with other reward functions using aggregated variables, the reward function 2 needs to calculate the waiting time of vehicles every time step. It leads to more training time for the agent to find the convergence direction. It also reflects that accurate vehicle data from the vehicular network may not be possible to improve the control performance of TSC tasks.

Table 3: The performance of the agent with different reward functions.

Reward functions	Traffic demand	Average delay (sec)	Average speed (km/h)	Total number of stops
1	Low	16.59 ± 0.41	25.60 ± 0.13	1283 ± 23
	High	35.70 ± 0.68	20.57 ± 0.14	3868 ± 63
2	Low	16.61 ± 0.37	25.60 ± 0.12	1283 ± 21
	High	35.70 ± 0.69	20.57 ± 0.14	3869 ± 63
3	Low	16.61 ± 0.37	25.60 ± 0.12	1283 ± 21
	High	31.71 ± 1.57	21.13 ± 0.37	3124 ± 130
4	Low	16.82 ± 0.39	25.56 ± 0.10	1285 ± 13
	High	28.71 ± 1.21	22.52 ± 0.28	2394 ± 89

5.3. Impact of control algorithms

The above experimental results show that the reward function 4 is better than other rewards. Thus, in this section, we evaluate the performance of different DRL-based algorithms with this reward function. Note that the traffic demand is dynamic, including low and high traffic demand, during the training episodes. Fig. 2 demonstrates that the convergence speed of A2C is faster than other three value-based methods. The reason is that the A2C does not adopt the replay memory technique that requires a large amount of sample data for training. However, the performance of the A2C is worse than others. One reason for this result is that the action space of this study is discrete, but policy-based methods suit for the continuous control problems. Another reason is that without the replay memory buffer, the critic network for the A2C algorithm is hard to learn useful advantage function, leading to the actor network incapable to output optimal control policy. Also, the DDQN and Dueling DQN perform better than the DQN. It indicates that two Q-functions and dueling network framework could help the agent learn better policies. Although the training curve of the DDQN and Dueling DQN converge to the similar value, the learning speed of Dueling DQN is faster than the DDQN.

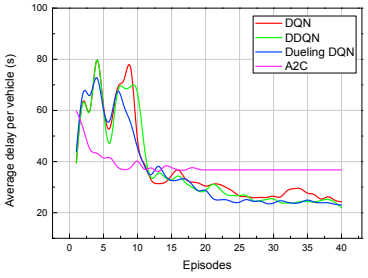


Fig. 2: The training process of different DRL-based algorithms

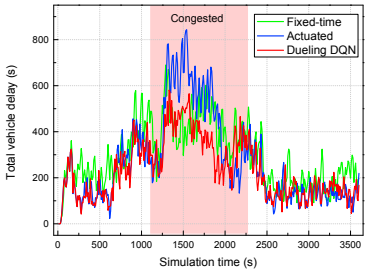


Fig. 3: Total vehicle delay of Dueling DQN and benchmarks

Table 4: The performance of DRL-based algorithms and benchmarks.

Metric	DQN	DDQN	Dueling DQN	A2C	Fixed-time	Actuated
Average reward	-222.76	-213.38	-211.22	-278.64	-	-
Average delay (sec)	25.06	24.03	**23.30**	36.76	40.17	31.03
Average speed (km/h)	23.25	23.51	**23.71**	20.61	19.93	21.82
Total number of stops	4547	4345	4325	5272	5446	**4074**

Table 4 shows the average performance of DRL-based algorithms and benchmarks. The performance of three value-based DRL algorithms is much better than traditional methods. Compared with the fixed-time control, Dueling DQN reduces about 42% delay, 21% total number of stops and increases about 20% vehicle speed. It also improves 24% in average delay compared with the actuated control method. However, the A2C algorithm has no improvement in reducing travel delay and increasing vehicle speed compared with the actuated method. It is worth noting that although the value-based DRL algorithms reduce the vehicle delay and increase speed, they do not reduce the total number of stops compared with the actuated control method. This is because the actuated control method tends to extend the green time to guarantee that vehicles controlled by the current phase pass through the intersection and ignore vehicles controlled by other phases. Fig. 3 further demonstrates the performance of the Dueling DQN and traditional methods

at each simulation step. It is worth noting that under the non-congested scenario, the Dueling DQN has a very similar performance to the actuated control method. However, the Dueling DQN outperforms traditional methods under the congested scenario. This finding suggests that designing a hybrid TSC method that combines with advantages of different control methods for different scenarios could be an interesting research direction.

6. Conclusion

In this study, we explore the impacts of different state definitions and reward functions on the TSC performance at an isolated intersection. Also, we investigate the training process of different DRL-based methods and compare their performances with traditional TSC methods. The experimental results show that the quantitative and image-like state definitions have no significant impact on TSC performance under different traffic demands. However, the impact of the reward functions is obvious, especially under high-demand scenarios. Moreover, we found that low-resolution traffic data is enough to guarantee the performance of DRL-based controllers. Thus, it is unnecessary in practice to pursue the perfect performance of DRL-based controllers to replace ordinary traffic devices with high-quality and high-resolution traffic sensors. In addition, the value-based DRL algorithms outperform the policy-based DRL algorithm and traditional TSC methods in terms of reducing average vehicle delay and improving speed.

Future interesting research directions include extending the experiments to road networks with multiple intersections to investigate the impacts of various design elements on the TSC performance. In addition to state definitions, reward functions, and DRL algorithms, it is also necessary to analyze the impacts of other design parameters, such as action space and hyper-parameters of DRL algorithms, on the TSC performance.

Acknowledgements

This work was supported by the National Research Foundation of Korea (NRF) grant funded by the Korea government (MSIT) (No. 2021R1C1C1006405).

References

[1] National Transportation Operations Coalition. (2012). National traffic signal report card.
[2] Wei, H., Zheng, G., Gayah, V., & Li, Z. (2019). A survey on traffic signal control methods. *arXiv preprint arXiv:1904.08117*.
[3] Arulkumaran, K., Deisenroth, M. P., Brundage, M., & Bharath, A. A. (2017). Deep reinforcement learning: A brief survey. *IEEE Signal Processing Magazine*, **34**(6), 26-38.
[4] El-Tantawy, S., Abdulhai, B., & Abdelgawad, H. (2014). Design of reinforcement learning parameters for seamless application of adaptive traffic signal control. *Journal of Intelligent Transportation Systems*, **18**(3), 227-245.
[5] Wei, H., Zheng, G., Yao, H., & Li, Z. (2018, July). Intellilight: A reinforcement learning approach for intelligent traffic light control. In *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining* (pp. 2496-2505).
[6] Abdulhai, B., Pringle, R., & Karakoulas, G. J. (2003). Reinforcement learning for true adaptive traffic signal control. *Journal of Transportation Engineering*, **129** (3).
[7] Wu, C., Ma, Z., & Kim, I. (2020, September). Multi-agent reinforcement learning for traffic signal control: Algorithms and robustness analysis. In *2020 IEEE 23rd International Conference on Intelligent Transportation Systems (ITSC)* (pp. 1-7). IEEE.
[8] Haydari, A., & Yilmaz, Y. (2020). Deep reinforcement learning for intelligent transportation systems: A survey. *IEEE Transactions on Intelligent Transportation*.
[9] Wei, H., Zheng, G., Gayah, V., & Li, Z. (2021). Recent advances in reinforcement learning for traffic signal control: A survey of models and evaluation. *ACM SIGKDD Explorations Newsletter*, **22**(2), 12-18.
[10] Farazi, N. P., Zou, B., Ahamed, T., & Barua, L. (2021). Deep reinforcement learning in transportation research: A review. *Transportation research interdisciplinary perspectives*, 11, 100425.
[11] Liang, X., Du, X., Wang, G., & Han, Z. (2019). A deep reinforcement learning network for traffic light cycle control. *IEEE Transactions on Vehicular Technology*, **68**(2), 1243-1253.
[12] Sutton, R. S., & Barto, A. G. (2018). *Reinforcement learning: An introduction*. MIT press.
[13] Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A. A., Veness, J., Bellemare, M. G., ... & Hassabis, D. (2015). Human-level control through deep reinforcement learning. *nature*, **518**(7540), 529-533.
[14] Van Hasselt, H., Guez, A., & Silver, D. (2016, March). Deep reinforcement learning with double q-learning. In *Proceedings of the AAAI conference on artificial intelligence* (Vol. 30, No. 1).
[15] Wang, Z., Schaul, T., Hessel, M., Hasselt, H., Lanctot, M., & Freitas, N. (2016, June). Dueling network architectures for deep reinforcement learning. In *International conference on machine learning* (pp. 1995-2003). PMLR.
[16] Ptv, A. G. (2018). PTV Vissim 10 user manual. *PTV AG: Karlsruhe, Germany.*